minicut Video Editor

Embedded LuaJIT & Plugin API Reference Manual
LuaJIT 5.1 Runtime Dockable Tool Panels Tool Shelf Dock Network Library (libcurl) FreeType Text Generation

1. Introduction & Architecture

minicut features a fully embedded, high-performance **LuaJIT** scripting engine. minicut enables automation, timeline editing, clip keyframing, and custom interactive dockable tool panels.

Script Directory Locations

Scripts and icons are discovered in the user's configuration directory:

Location	Trigger / Role	Description
`~/.config/minicut/autorun/`	Startup / Auto-register	Scripts execute on startup and generate an icon button on the horizontal Tool Shelf Dock below the main menu bar.
`~/.config/minicut/scripts/`	Menu Click / On Demand	Scripts appear in the top-level **Scripts** menu on the main menu bar. Clicking runs the script and opens its tool panel.
`~/.config/minicut/icons/`	Dock Icon Asset	32x32 PNG icon images matching script stems (e.g., `quick_placement.png` for `quick_placement.lua`).

2. Plugin Script Architecture

Every minicut script defines timeline and clip manipulation callbacks, followed by a `plugin:init_panel()` method that registers Dear ImGui UI controls.

```
-- ========================================================================
-- Example Base Script: my_tool.lua
-- ========================================================================

local target_track = 0
local clip_scale = 1.25

-- Timeline and clip manipulation logic
function execute_placement()
    minicut.push_undo()
    local path = "/path/to/media.mp4"
    local asset_id = project.import_video(path)
    local clip_id = timeline.place_video(target_track, asset_id,
timeline.get_playhead(), 5.0)
    if clip_id ~= "" then
        clip.set_scale(clip_id, clip_scale)
        timeline.add_keyframe(clip_id, "opacity", 0.0, 0.0)
        timeline.add_keyframe(clip_id, "opacity", 1.0, 1.0)
        minicut.set_status("Placed media clip on track " .. target_track)
    end
end
```

```
-- GUI panel declaration
function plugin:init_panel()
    gui.text("Media Placement Tool")
    gui.separator()
    target_track = gui.slider_int("Track Lane", target_track, 0, 4)
    clip_scale = gui.slider_float("Initial Scale", clip_scale, 0.1, 5.0)

    if gui.button("Place Clip At Playhead", execute_placement) then
        -- Optional inline handling
    end
end
```

3. Core Lua Modules

3.1. Project & File Automation (`project` / Global)

Function	Returns	Description
`project_save([path])` `project.save([path])`	`boolean, string`	Saves current project state to a `.minicut` JSON file.
`project_load(path)` `project.load(path)`	`boolean, string`	Loads a project from `path`, refreshing timeline and media.
`project.get_name()`	`string`	Returns the current project name.
`project.set_name(name)`	`void`	Sets the current project name.
`project.import_video(path)`	`string (asset_id)`	Imports video into the project media pool.
`project.import_audio(path)`	`string (asset_id)`	Imports audio into the project media pool.
`project.import_image(path, [dur])`	`string (asset_id)`	Imports image into the project media pool (default: 5.0s).
`assets.create_text(text, [opts])`	`string (asset_id)`	Generates FreeType text asset with font, outline, and alignment.

3.2. Network & Internet Library (`net`)

Function	Returns	Description
`net.get(url)`	`table`	HTTP GET request. Returns `{{status, body, ok, headers}}`.
`net.post(url, body)`	`table`	HTTP POST request with string or JSON payload.
`net.download(url, dest)`	`boolean`	Downloads remote asset directly to local disk.
`net.upload(url, path)`	`table`	HTTP POST/upload local file binary stream.

Function	Returns	Description
`net.request(options)`	`table`	Custom HTTP request (url, method, headers, timeout).

3.3. Timeline & Track Management (`timeline`)

Function	Returns	Description
`timeline.get_playhead()`	`number`	Current playhead position in seconds.
`timeline.set_playhead(t)`	`void`	Moves timeline playhead to `t` seconds.
`timeline.get_duration()`	`number`	Total duration of the project timeline.
`timeline.get_track_count()`	`integer`	Total number of video and audio tracks.
`timeline.add_track(type, [name])`	`integer`	Adds a track ("video" or "audio") with optional name.
`timeline.place_video(track, id, start, dur)`	`string`	Places video clip on track and returns clip ID.
`timeline.place_audio(track, id, start, dur)`	`string`	Places audio clip on track and returns clip ID.
`timeline.place_image(track, id, start, dur)`	`string`	Places image clip on track and returns clip ID.
`timeline.place_text_clip(track, text, start, dur)`	`string`	Generates and places text clip on track.
`timeline.delete_clip(clip_id)`	`boolean`	Deletes clip and removes its associated keyframes.
`timeline.split_clip(clip_id, [t])`	`boolean`	Splits clip at time `t` (defaults to playhead).
`timeline.select_clips(ids_table)`	`void`	Selects multiple clips simultaneously.
`timeline.get_selected_clips()`	`table`	Returns array of selected clip IDs.
`timeline.clear_selection()`	`void`	Clears timeline selection.
`timeline.move_track_up(idx)`	`void`	Moves track up in stack order.
`timeline.move_track_down(idx)`	`void`	Moves track down in stack order.

Function	Returns	Description
`timeline.set_track_name(idx, name)`	`void`	Renames track at index.
`timeline.get_track_name(idx)`	`string`	Gets track name at index.
`timeline.add_keyframe(id, prop, t, val)`	`void`	Adds animation keyframe (opacity, volume, pos_x, etc.).
`timeline.remove_keyframe(id, prop, t)`	`void`	Removes keyframe near timestamp `t`.
`timeline.evaluate_keyframe(id, prop, t, [def])`	`number`	Returns linearly interpolated parameter value.

3.4. Clip Transformations (`clip`)

Function	Returns	Description
`clip.get_position(clip_id)`	`number, number`	Returns (pos_x, pos_y) relative to canvas center.
`clip.set_position(clip_id, x, y)`	`void`	Sets center position offset in pixels.
`clip.get_scale(clip_id)`	`number`	Returns uniform scale multiplier (1.0 = 100%).
`clip.set_scale(clip_id, s)`	`void`	Sets uniform scale multiplier.
`clip.get_rotation(clip_id)`	`number`	Returns rotation angle in degrees.
`clip.set_rotation(clip_id, r)`	`void`	Sets rotation angle in degrees.
`clip.get_opacity(clip_id)`	`number`	Returns opacity multiplier [0.0, 1.0].
`clip.set_opacity(clip_id, o)`	`void`	Sets opacity multiplier.
`clip.get_volume(clip_id)`	`number`	Returns audio volume multiplier [0.0, 2.0].
`clip.set_volume(clip_id, v)`	`void`	Sets audio volume multiplier.
`clip.get_time(clip_id)`	`number, number, number`	Returns (start_time, duration, offset).
`clip.set_time(clip_id, start, dur, [off])`	`void`	Modifies clip timeline timing and media offset.
`clip.get_info(clip_id)`	`table`	Returns comprehensive metadata

Function	Returns	Description
		dictionary.

3.5. GUI & UI Elements (`gui`)

Function	Returns	Description
`gui.text(text)`	`void`	Displays a standard text string.
`gui.text_colored(r, g, b, a, text)`	`void`	Displays colored text with RGBA float components.
`gui.button(label, [callback])`	`boolean`	Renders push button, returns true if clicked.
`gui.slider_float(label, val, min, max, [cb])`	`number`	Renders horizontal float slider.
`gui.slider_int(label, val, min, max, [cb])`	`integer`	Renders integer slider.
`gui.drag_float(label, val, [spd, min, max, cb])`	`number`	Renders click-and-drag float value editor.
`gui.input_text(label, text, [cb])`	`string`	Single-line text entry input box.
`gui.checkbox(label, state, [cb])`	`boolean`	Toggle checkbox.
`gui.same_line()`	`void`	Positions next item on the same line horizontally.
`gui.separator()`	`void`	Draws horizontal divider rule.
`gui.file_dialog(options)`	`void`	Opens file dialog matching user settings (native or built-in).

3.6. Engine Utilities (`minicut`)

Function	Returns	Description
`minicut.push_undo()`	`void`	Pushes current project state onto undo history stack.
`minicut.set_status(msg, [dur])`	`void`	Displays temporary status toast notification.
`minicut.log(msg)`	`void`	Outputs debug line to standard console output.